

Autonomous Obstacle-Avoiding Robot

One of the last project that I worked on over the summer of 2026 was a 4WD robot built that was built on an Arduino UNO that drives forward autonomously, detects obstacles using an ultrasonic sensor mounted on a sweeping servo, and intelligently decides which direction to turn based on which way is the best path. The robot determines the best “path” by scanning the area with the ultrasonic sensors and it finds the path with the largest distances since that would essentially mean that that would be the most clear path available. This is much better than just a robot that does the same turn over and over again until a new path occurs since it is much more efficient. I built this project on what I learned with my Ultrasonic radar system I built before this project and I reused the same sensor and servo components to build this robot and modified some of the code.

How it works

When the robot is active it will try to drive forward first then when it detects an object or anything in its direct path within ~32 cms it will stop then:

1. Backs up briefly to create scanning room
2. Sweeps the servo-mounted sensor from 5° to 175°, taking a distance reading at every 5° step
3. Compares every reading to find the angle with the most open space
4. Pivots toward that direction
5. Re-centers the sensor and resumes driving forward

A pushbutton toggles the whole system on/off, so it doesn't start driving when the instant power is connected.

Hardware used

- Arduino Uno
- 4WD acrylic chassis (4x TT gear motors, wheels)
- L298N dual H-bridge motor driver
- HC-SR04 ultrasonic sensor
- SG90 servo motor
- Pushbutton (on/off toggle)
- Battery pack (motor + logic power)

Software used

- Arduino IDE (C / C++)

What I learned

- Driving DC motors through an H-bridge motor driver (L298N), including how a single dual-channel driver can control 4 motors by bridging pairs in parallel
- Building a simple state machine in C to switch a device between distinct behaviors (driving vs. scanning) based on sensor input
- Using a `for` loop to run a self-contained servo sweep, tracking a "best value found so far" using two persistent variables (`bestDistance`, `bestAngle`) — the same comparison pattern used in searching/optimization problems generally
- Debugging real hardware issues independently: reversed motor direction on a symmetrically-mounted chassis, a pin conflict between a servo library and motor control pins, shared power/ground requirements across multiple modules, and a mechanically defective motor gearbox

Challenges and how I solved them

- **Incorrect motor shield.** When ordering the components for the robot I had to get a motor shield so I could connect and control all the motors that are connected to the wheels. I should have used a L293D motor shield since it would have been a lot easier to connect and since I have 4 separate motors, one for each wheel, the motor shield also supports 4 motors repeatedly. However I ordered the L293N motor shield not knowing the L293D was expedited. This wasn't a big issue but it made the wiring a bit tricky since the shield only supported two motor shields separately and I needed to support four. So I had to work out the correct wiring and direction logic from the L298N's own pinout.
- **Motors mounted in a rotationally symmetric pattern.** Diagonal motor pairs faced the same physical direction rather than a simple left/right mirror, so matching wire colors (red-to-red) didn't guarantee both motors on a side spun the same way. Solved by wiring based on color first, then testing each side in isolation and swapping terminals wherever a motor spun opposite its partner.
- **One motor's gearbox is mechanically defective** — it spins but doesn't properly transfer motion to the wheel, confirmed by comparing gear resistance/sound by hand against the other three motors.
- **Stale sensor data bugs.** Several early versions read `distance` before the sensor had actually taken a new measurement that loop, using leftover data from the previous cycle. Fixed by reordering code so every measurement happens immediately before it's used.
- **Mid-scan turning bug.** An early version placed the "turn toward best angle" logic inside the scanning `for` loop instead of after it, causing the robot to react to a still-changing, not-yet-final `bestAngle` on every single step of the scan instead of once at the end.

Future improvements

- Add variable motor speed control via the L298N's ENA/ENB pins (currently jumpered to always-full-speed)
- Add line-tracking sensors for a second, selectable driving mode
- Refactor blocking `delay()` calls to non-blocking timing using `millis()`, so the robot can be more responsive during scans